

INTERNATIONAL SCHOOL, VIETNAM NATIONAL
UNIVERSITY
COMPUTER VISION

FINAL REPORT

A Comparative Performance Analysis of CNN-based and Transformer-based Architectures for PPE Detection in Real-Time IoT Systems

Instructor: Kim Dinh Thai
Course section: INS315501
Group: Group 2
Submission date: 7/6/2026

Hanoi, 2026

CONTRIBUTION

No.	Full Name	Student ID	Assigned Tasks	Contribution
1	Doan Duy Long	22070843	- Developed and fine-tuned the YOLOv8 Anchor-Free baseline model. - Engineered the complete closed-loop IoT alert framework for real-world production monitoring. - Built the Edge Vision Processing Node using Streamlit, integrated Cloud Telemetry via Telegram API, and programmed the physical hardware actuation using MQTT and an ESP32 microcontroller.	33.3%
2	Pham Tan Minh	22070977	- Researched and implemented the YOLO26 model, exploring its End-to-End NMS-Free paradigm to maximize hardware throughput. - Handled the Roboflow PPE dataset preparation, including standardized spatial preprocessing 640x640 and Architectural-Aware dynamic augmentations. - Extracted and structured the evaluation metrics (mAP, Precision, Recall) and generated the qualitative visual prediction outputs for model comparison.	33.3%
3	Nguyen Tran Hai Phong	22070701	- Researched and implemented the complex RT-DETR (Vision Transformer) architecture, focusing on Self-Attention mechanisms and Cross-scale feature fusion. - Conducted the rigorous Error Analysis to identify failure cases under extreme occlusions and class ambiguity. - Handled the mathematical evaluation of computational efficiency across all models (calculating FLOPs, parameter footprints, and FPS trade-offs).	33.3%

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our instructor for the continuous guidance, constructive feedback and dedicated support provided throughout the development and completion of this research project. The theoretical knowledge and practical skills acquired during this course have been instrumental in helping us systematically execute a rigorous experimental study, encompassing problem formulation, dataset preprocessing, model evaluation and scientific reporting.

We also gratefully acknowledge the open-source community and the developers of the PyTorch and Ultralytics frameworks for providing the foundational software tools necessary for our model implementations. Special thanks are extended to Roboflow Universe for providing the specialized Personal Protective Equipment (PPE) dataset that made this empirical study possible. Additionally, we appreciate the cloud computational resources provided by Google Colab, which significantly facilitated our model training and evaluation phases. All external resources, source codes, datasets and references utilized in this report have been appropriately acknowledged and cited.

ABSTRACT

This study addresses the real-time object recognition problem in the context of occupational safety monitoring, specifically checking compliance with the use of Personal Protective Equipment (PPE). The goal of the study is to evaluate the trade-off between prediction accuracy and computational performance among different deep learning architectures. Experiments were conducted on an industrial image dataset consisting of 7 object classes, strictly divided into training, validation and testing sets. Three models (YOLOv8, YOLO26 and RT-DETR) were comprehensively compared based on both performance metrics (mAP, Precision, Recall) and computational cost metrics (number of parameters, FLOPs, FPS). Experimental results showed that the RT-DETR model achieved the highest accuracy (mAP@50-95 is 0.5279) but was limited in speed (25.29 FPS). In contrast, the YOLO26 achieves an optimal balance with competitive accuracy (mAP@50-95 is 0.4925) while maintaining superior inference speed (60.07 FPS) with a size of only 2.51 million parameters. These findings indicate that lightweight architectures are perfectly suited for deployment in real-world surveillance systems where demanding requirements for both accurate detection and real-time processing capabilities are paramount.

Keywords: deep learning, object detection, real-time inference, lightweight model, visual inspection.

Contents

CONTRIBUTION	5
ACKNOWLEDGEMENTS	5
ABSTRACT	5
List of Figures	6
List of Tables	7
1 INTRODUCTION	8
2 MATERIALS AND METHODS	10
2.1 Dataset and Materials	10
2.2 Standardized Data Preprocessing and Architectural-Aware Augmentation . .	10
2.2.1 Base Preprocessing Protocol	11
2.2.2 Dynamic Data Augmentation Strategy	11
2.2.3 Architectural-Specific Adjustments (CNN vs. Transformer)	11
2.3 Model Architecture	12
2.3.1 Design Rationale	15
2.3.2 Loss Function and Optimization Objective	16
2.3.3 Training and Inference Algorithm	17
2.4 Experimental Setup	18
2.5 Evaluation Metrics	19
2.5.1 Detection Performance Metrics	19
2.5.2 Computational Efficiency Metrics	20
3 RESULTS	22
3.1 Quantitative Comparison	22
3.2 Computational Efficiency and Hardware Trade-offs	23
3.3 Qualitative Results	23
3.4 In-depth Error Analysis	24
4 ADDITIONAL STUDY : Production-Ready Edge Deployment and IoT Integration	26
4.1 Edge Vision Processing Node (Python & Streamlit)	26
4.2 Cloud Telemetry and Remote Monitoring (Telegram API)	26
4.3 Physical Hardware Actuation (MQTT and ESP32 Microcontroller)	27

5	DISCUSSION	29
6	CONCLUSION	30

List of Figures

1	Software execution evidence showing the validation log and terminal outputs of the YOLO26 model.	22
2	Qualitative comparison of detection outputs across varying visual conditions (normal, crowded, and small objects).	24
3	Representative failure cases highlighting contextual dependency flaws and class ambiguity in isolated PPE scenarios.	24
4	System process block diagram illustrating the data flow from the edge vision node to cloud telemetry and physical ESP32 hardware actuation.	27
5	Real-world deployment evidence. (A) Edge node interface detecting a PPE violation. (B) Zero-latency remote notification via Telegram. (C) Physical actuation triggered via the ESP32 microcontroller.	27

List of Tables

1	Summary of the dataset used in this study.	10
2	Main training and implementation settings.	19
3	Performance comparison on the test set.	22

1 INTRODUCTION

Automated visual inspection has become a core research direction in industrial safety management. This method helps minimize dependence on manual labor, increases objectivity and continuity in monitoring and supports real-time warning decision-making to prevent occupational accidents [1]. However, in real-life construction site or factory environments, identification of Personal Protective Equipment (PPE) faces harsh visual challenges. The complex changes in lighting conditions, extremely small objects (such as goggles, gloves), and occlusion in crowds make feature extraction very difficult.

To solve the above challenges, recent research has strongly applied deep learning methods. These approaches range from using convolutional neural networks (CNN) with anchor-free design to improve PPE localization [2], to applying models based on Transformer architecture to exploit self-attention mechanisms for better recognition in obscured environments [3]. In particular, one-stage detection frameworks such as the YOLO model family have provided a very good balance between accuracy and speed [4]. However, the reality is that the models with the highest accuracy (such as DETR or Swin-Transformer) require huge computational costs, making them almost impossible to deploy on edge devices with low resources and limited energy.

Despite these technological advancements, several critical research gaps persist in the field of PPE identification. First, many existing studies prioritize the optimization of detection accuracy metrics-such as mAP, Precision, and Recall-while neglecting the evaluation of essential computational metrics for practical deployment, including parameter counts, FLOPs, and inference speed (FPS). Second, current research often focuses on refining and independently evaluating a single model architecture, resulting in a lack of comprehensive benchmarking platforms that allow for a direct comparison of the relative strengths and weaknesses between local CNN-based models and global Transformer-based architectures on identical datasets. Finally, qualitative analysis and the assessment of failure cases remain largely superficial, failing to provide sufficient insight into system performance when encountering small objects, such as goggles and gloves, or when dealing with environmental disturbances and complex backgrounds.

To fill these gaps, our research aims to cross-match state-of-the-art object detection models for the PPE recognition problem, thereby identifying a solution that provides the most optimal balance between prediction accuracy, computational cost, and applicability on limited hardware. The main contributions of this report are summarized as follows:

- (1) We define the problem of PPE identification and compliance checking as a supervised multi-layer object detection task, directly serving industrial safety warnings.
- (2) We prepare, preprocess images and establish a rigorous experimental protocol on a real construction site data set.

(3) We compare three representative models (YOLOv8n, NMS-free variant YOLO26n, and RT-DETR attention model) on the same evaluation platform; and transparently report both reference systems: prediction performance (mAP) and computational efficiency (Params, FLOPs, FPS).

(4) We perform in-depth analysis of quantitative results, compare computational complexity, provide qualitative examples and deeply dissect error identification cases (error analysis) to provide the most objective and comprehensive assessment.

2 MATERIALS AND METHODS

2.1 Dataset and Materials

To comprehensively evaluate the performance trade-offs among the four selected architectures ranging from fully convolutional networks (YOLOv8, YOLOv26) to attention-based Transformers (RT-DETR, D-Fine) this study utilizes a custom Personal Protective Equipment (PPE) dataset sourced from Roboflow Universe (Version 3). The dataset is specifically curated for automated workplace safety monitoring in dynamic industrial environments, where occlusions, varying scales of objects, and unpredictable lighting conditions heavily challenge object detection algorithms.

Unlike generalized open-source datasets, this specific industrial problem is strictly formulated as a 7-class detection task. The semantic classes are strategically grouped into three categories: compliant safety gear (Helmet, Goggles, Gloves), safety violations (No-Helmet, No-Goggles, No-Gloves), and contextual structural objects (Person). This highly granular class definition serves a dual purpose: first, it forces the neural networks to learn highly discriminative, fine-grained features for specific safety violations rather than generic backgrounds; second, it provides a rigorous testbed to evaluate how different architectures handle extreme class imbalances and small-object localization (e.g., distinguishing Goggles on a Person).

To ensure a rigorous and unbiased empirical benchmarking process, the total dataset of 9,968 images was strictly partitioned into training, validation and testing subsets. All architectures are trained and evaluated on this exact same data split to guarantee that the comparative metrics (mAP, Precision, Recall, FPS and FLOPs) are perfectly standardized and free from data leakage. The annotations are encoded in the standard YOLO normalized bounding box format $(x_{center}, y_{center}, w, h)$.

Subset	Number of Images	Number of Labels	Percentage
Training	8,160	35,238	~81.8%
Validation	905	3,819	~9.1%
Testing	903	3,823	~9.1%

Table 1: Summary of the dataset used in this study.

2.2 Standardized Data Preprocessing and Architectural-Aware Augmentation

Establishing a standardized preprocessing pipeline is one of the most critical steps when comparing models with fundamentally different inductive biases, such as CNNs and Transformers.

Different architectures react differently to spatial distortions and pixel-level noise; thus, the data pipeline in this study is carefully calibrated to ensure absolute empirical fairness.

2.2.1 Base Preprocessing Protocol

Before entering the neural networks, all input images undergo a unified preprocessing stage. Images are uniformly resized using a "Stretch to (640x640)" technique. This fixed spatial resolution acts as the common mathematical denominator across all experiments, providing an optimal trade-off: preserving sufficient fine-grained visual details required to detect small-scale objects (like Goggles or Gloves) while standardizing FLOPs calculation for objective computational complexity evaluation. Furthermore, pixel intensities are normalized from $[0, 255]$ to the $[0, 1]$ range to accelerate gradient descent convergence and maintain numerical stability across network layers.

2.2.2 Dynamic Data Augmentation Strategy

During the training phase, dynamic augmentations are applied on-the-fly to artificially expand the dataset, simulate real-world industrial anomalies, and mitigate overfitting. The baseline transformations include:

- Geometric Distortions: Random Rotation ($\pm 15^\circ$) and Shear ($\pm 15^\circ$) to simulate varied mounting angles and mechanical vibrations of field surveillance cameras.
- Photometric Adjustments: Random Brightness shifts ($\pm 23^\circ$), ToGray conversion, and CLAHE (Contrast Limited Adaptive Histogram Equalization) to ensure robust detection in overexposed or dimly lit environments.
- Image Degradation: Blur and MedianBlur (kernel limits 3-7) to simulate motion blur and sensor noise from low-quality CCTV streams.

2.2.3 Architectural-Specific Adjustments (CNN vs. Transformer)

A key scientific distinction in our training methodology addresses the fundamental architectural differences between the models regarding complex spatial augmentations, particularly Mosaic. Mosaic augmentation combines four distinct images into a single training grid, drastically enriching contextual background and multiplying the number of targets per batch.

- For CNN-based models (YOLOv8, YOLO26): Localized convolutional kernels benefit immensely from dense spatial distributions. Therefore, aggressive Mosaic augmentation is heavily utilized throughout their entire training cycle, enabling them to excel at detecting small-scale equipment like gloves and goggles.

- For Transformer-based models (RT-DETR): These architectures rely on global Self-Attention mechanisms and one-to-one set prediction via Hungarian matching. Extreme spatial mixing (like continuous Mosaic) destroys the natural global context of an image and can severely destabilize their Bipartite Matching Loss. Consequently, an early-stopping strategy is implemented for Mosaic augmentation: it is strictly disabled during the final 10 epochs of training for RT-DETR and D-Fine. This architectural-aware adjustment allows the fine-grained bounding box refinement process within the Transformer decoders to converge stably and optimally.

To maintain the absolute integrity of the final evaluation metrics, no augmentations whatsoever are applied to the validation and test sets.

2.3 Model Architecture

A. YOLOv8: The High-Accuracy Anchor-Free Baseline

YOLOv8 represents a significant paradigm shift in the YOLO family by completely abandoning anchor boxes in favor of an anchor-free detection mechanism.

- **Backbone and Neck Architecture** The architecture utilizes a modified CSPDarknet53 backbone integrated with novel C2f (Cross Stage Partial bottleneck with 2 convolutions) modules. The C2f module enriches gradient flow and captures multi-scale contextual features without exponentially increasing the parameter count. Furthermore, the PANet (Path Aggregation Network) neck ensures that semantic features are strongly fused across different spatial resolutions.
- **Decoupled Head and Loss Functions** Unlike earlier versions, YOLOv8 implements a decoupled head that physically separates the classification (c_i) and regression (b_i) tasks into two distinct convolutional branches, effectively resolving the inherent feature conflict between object categorization and localization. The network optimizes a multi-task loss function during backpropagation:

$$L_{v8} = \lambda_1 L_{VFL} + \lambda_2 L_{CIoU} + \lambda_3 L_{DFL} \quad (1)$$

Key components of the loss function include:

- **Varifocal Loss (L_{VFL}):** Replaces standard Binary Cross-Entropy by asymmetrically weighting positive and negative samples, allowing the network to focus strictly on high-quality bounding boxes while suppressing background noise.
- **Complete IoU Loss (L_{CIoU}):** A sophisticated regression loss that minimizes three geometric factors simultaneously: the overlap area, the normalized distance between center points, and the consistency of the aspect ratio (v).

- Distribution Focal Loss (L_{DFL}): Rather than treating bounding box edges as rigid Dirac delta functions, L_{DFL} models box boundaries as a general probability distribution, enabling effective localization of objects with blurred or occluded edges (e.g., a partially hidden helmet).

The NMS Bottleneck: : Despite its high accuracy (mAP), YOLOv8 relies heavily on Non-Maximum Suppression (NMS) to filter thousands of overlapping candidate bounding boxes during inference. Since NMS operates sequentially ($O(N^2)$) and requires intense logical branching on the CPU, it creates a significant computational bottleneck when deployed on resource-constrained microcontrollers or IoT edge devices.

B. YOLO26: The Edge-Optimized, NMS-Free Architecture

YOLO26 is specifically engineered to eradicate the latency bottlenecks of its predecessors, targeting seamless deployment on resource-constrained industrial edge hardware. The architectural breakthrough of YOLO26 is its end-to-end NMS-free paradigm and the complete removal of the heavy DFL module.

Rather than predicting redundant boxes and relying on post-processing, YOLO26 embeds the filtering mechanism directly into the training phase via Bipartite Matching (inspired by the Hungarian algorithm). It establishes a strict one-to-one matching between ground truth objects and model predictions. The bipartite matching cost C_{match} between a ground truth y_i and a prediction $\hat{y}_{\sigma(i)}$ is formulated as:

$$C_{match} = \lambda_1 L_{cls}(c_i, \hat{p}_{\sigma(i)}) + \lambda_2 L_{L1}(b_i, \hat{b}_{\sigma(i)}) + \lambda_3 L_{GIoU}(b_i, \hat{b}_{\sigma(i)}) \quad (2)$$

The network utilizes the Hungarian algorithm to find the optimal permutation matrix σ that minimizes this global assignment cost :

$$\hat{\sigma} = \arg \min_{\sigma \in \Sigma N} \sum_{i=1}^N C_{match}(y_i, \widehat{y}_{\sigma(i)})$$

By applying severe mathematical penalties to redundant bounding boxes during training, the network learns to natively output exactly one optimal box per object. Consequently, the entire NMS post-processing block is physically stripped from the inference code. This eliminates memory-intensive sorting operations and CPU branching loops, heavily accelerating the FPS (Frames Per Second) rate and making YOLOv26 the premier candidate for direct integration with real-time alerting hardware like the ESP32.

C. RT-DETR

RT-DETR (Real-Time DEtection Transformer) marks a significant shift in its approach to pure active neural network (CNN) architectures through the integration of a self-attention mechanism. The core of this architecture lies in a specialized design to capture spatial dependencies over long distances (long-range spatial dependencies) and understand the global

context of the image. This allows the model to logically infer relationships between objects ,for example : easily associating a safety helmet with a worker even if they are partially obscured in the frame.

To achieve optimal performance, RT-DETR’s architecture is built upon a series of ground-breaking, tightly interconnected components. First, to address the computational complexity barrier of exponentially proportional features ($O(N^2)$) inherent in traditional Vision Transformer models, which stifles real-time processing capabilities, RT-DETR employs an Efficient Hybrid Encoder. This design cleverly separates feature interaction within the same scale from cross-scale feature fusion. Specifically, the AIFI (Attention-based Intra-scale Feature Interaction) module is applied only to the highest-level semantic feature layers to minimize computation length. These features are then seamlessly combined via the CCFM (Cross-scale Convolution-fusion Module), allowing for smooth blending of multi-scale features without overloading memory bandwidth.

The data stream, after passing through the encoder, is received by the decoder. Here, instead of scanning the image with a dense and rigid network of convolutional anchors like older architectures, RT-DETR uses a fixed number of learnable object queries. Thanks to the cross-attention mechanism, these queries continuously searching feature maps, thereby learning to accurately attend to the areas with the highest probability of object presence.

Thanks to this flexible querying method, RT-DETR can approach object discovery as a direct set prediction problem, thereby completely eliminating the bottleneck of the NMS post-processing algorithm. To achieve this, the model uses a Bipartite Matching algorithm to force a tight one-to-one mapping between object queries and the actual label . This mapping optimization process is driven by an overall attenuation function, which is a linear combination of Focal Loss for classification and bounded box regression attenuation functions:

$$L_{RTDETR} = \lambda_{cls} L_{Focal}(c, \hat{c}) + \lambda_{L1} L_{L1}(b, \hat{b}) + \lambda_{GIoU} L_{GIoU}(b, \hat{b})$$

In the above equation, L_{Focal} plays a core role in minimizing extreme class imbalance. Simultaneously, the combination of L_{L1} and L_{GIoU} ensures absolute accuracy of the bounding box coordinates along with geometric uniformity, despite significant changes in the scaling of objects within the image.

While this groundbreaking NMS-free design makes the end-to-end inference flow seamless, it imposes significant impacts on hardware requirements. The enormous volume of Multi-Head Attention blocks—comprising approximately 32 million parameters—places immense pressure on memory bandwidth and computational power (FLOPs).

Consequently, while RT-DETR achieves the highest absolute accuracy (mAP) and exceptional resilience to false negatives in crowded environments, this superiority comes at the

cost of inference speed (approximately 25 FPS compared to 60 FPS in YOLO-based models). Due to this heavy yet precise nature, RT-DETR relies heavily on high-throughput GPUs or Tensor core accelerators, establishing itself as the optimal choice for highly reliable centralized security monitoring systems, rather than for deployments on resource-limited edge microcontrollers.

2.3.1 Design Rationale

The selection of YOLOv8, YOLO26, and RT-DETR is deliberately designed to evaluate the evolutionary spectrum of real-time object detection. The objective is to analyze the fundamental trade-offs between Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs), specifically focusing on their viability for edge-deployment and hardware-in-the-loop integration.

- **YOLOv8 (The Industry Baseline):** YOLOv8 is selected as the standard reference benchmark. Its robust anchor-free architecture and decoupled head provide a highly reliable baseline for evaluating Mean Average Precision (mAP) versus Frames Per Second (FPS). However, its inherent reliance on the Non-Maximum Suppression (NMS) post-processing algorithm creates a known CPU bottleneck, serving as the control variable to measure the latency improvements of subsequent NMS-free models.
- **YOLO26 (The Edge-Optimized CNN):** YOLO26 is chosen to represent the cutting-edge advancement within the CNN paradigm. By integrating Bipartite Matching via the Hungarian algorithm, YOLO26 fundamentally eliminates the NMS module. This model is evaluated as the primary candidate for maximizing hardware throughput, aiming to prove that removing memory-intensive sorting operations drastically reduces I/O latency when communicating with constrained microcontrollers like the ESP32.
- **RT-DETR (The Transformer Pioneer):** RT-DETR is introduced to contrast the local feature extraction of CNNs with the global contextual reasoning of the Self-Attention mechanism. Like YOLO26, it is NMS-free, but it utilizes transformer encoder-decoder layers to achieve superior accuracy, particularly in complex industrial environments with occluded PPE. RT-DETR serves as the "accuracy ceiling" in this research, determining whether the higher computational cost (FLOPs) of Transformers is justified when balanced against the networking and actuation demands of an IoT safety system.

By benchmarking these three distinct architectures, this study provides a comprehensive roadmap for selecting the optimal inference engine that bridges high-accuracy computer vision with zero-latency physical hardware actuation.

2.3.2 Loss Function and Optimization Objective

Since this study compares three object detection architectures (YOLOv8, YOLO26, and RT-DETR), each model employs a different optimization objective while sharing the same goal: accurately localizing PPE objects and correctly classifying their categories.

A. YOLOv8 Loss Function YOLOv8 adopts a multi-task optimization objective consisting of classification, localization, and distribution refinement losses:

$$L_{\text{YOLOv8}} = \lambda_1 L_{\text{VFL}} + \lambda_2 L_{\text{CIoU}} + \lambda_3 L_{\text{DFL}}$$

where L_{VFL} denotes the Varifocal Loss for classification, L_{CIoU} represents the Complete-IoU loss for bounding-box regression, and L_{DFL} is the Distribution Focal Loss used for fine-grained localization. The Complete-IoU loss is expressed as:

$$L_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2} + \alpha v$$

where L_{cls} measures classification errors, L_{L1} penalizes coordinate regression errors, and L_{GIoU} evaluates geometric consistency between predicted and ground-truth bounding boxes.

B. YOLO26 Loss Function

YOLO26 follows an End-to-End NMS-Free paradigm. Instead of generating redundant predictions and removing them using Non-Maximum Suppression (NMS), the model directly learns a one-to-one assignment between predictions and ground-truth objects through Bipartite Matching. The matching cost is defined as:

$$C_{\text{match}} = \lambda_1 L_{\text{cls}} + \lambda_2 L_{L1} + \lambda_3 L_{\text{GIoU}}$$

where:

- L_{cls} measures classification errors.
- L_{L1} penalizes coordinate regression errors.
- L_{GIoU} evaluates geometric consistency between predicted and ground-truth bounding boxes.

C. RT-DETR Loss Function

RT-DETR formulates object detection as a direct set prediction problem. Similar to DETR-based architectures, the model employs Bipartite Matching combined with classification and localization losses. The overall optimization objective is:

$$L_{\text{RT-DETR}} = L_{\text{Focal}} + \lambda_1 L_{L1} + \lambda_2 L_{\text{GIoU}}$$

where:

- L_{Focal} addresses class imbalance during classification.
- L_{L1} minimizes coordinate prediction errors.
- L_{GIoU} improves localization quality and bounding-box overlap.

The Bipartite Matching strategy guarantees a unique correspondence between object queries and ground-truth labels. Consequently, RT-DETR avoids the need for post-processing NMS and enables fully end-to-end optimization.

Overall, all three architectures are trained by minimizing their corresponding objective functions through gradient-based optimization. The loss functions jointly optimize classification accuracy, localization precision, and robustness against challenging PPE detection scenarios such as small objects, occlusions, and crowded environments.

2.3.3 Training and Inference Algorithm

This algorithm summarizes the end-to-end pipeline for training, validating, and evaluating the object detection models. Training and Inference Pipeline

Input: PPE Dataset, selected model, learning rate, and number of epochs.

Output: Best trained model and final evaluation metrics.

1. Data Preparation:

- Split the dataset into training, validation, and testing subsets.
- Resize inputs to 640×640 , normalize pixel values, and apply data augmentations strictly to the training set.

2. Training Phase:

For each *epoch* do:

For each *mini-batch* in the training set do:

- * Compute model predictions and calculate the objective loss.
- * Update the model weights using backpropagation.
- Evaluate the model on the validation set.
- Save the model checkpoint if the validation mAP@0.50:0.95 improves.

3. Inference & Evaluation Phase:

- Load the best-performing model checkpoint and run inference on the unseen test set.
- Perform Post-processing:
 - **YOLOv8:** Apply confidence thresholding followed by Non-Maximum Suppression (NMS).
 - **YOLO26 & RT-DETR:** Apply confidence thresholding only (End-to-End NMS-Free).
- Calculate quantitative metrics (mAP, Precision, Recall, FPS, Parameters) and extract qualitative failure cases for error analysis.

4. Return final predictions and evaluation results.

Description: This standardized workflow ensures fair evaluation across all architectures. The post-processing step explicitly highlights a critical architectural shift: transitioning from YOLOv8’s computationally heavy NMS heuristic to the highly optimized, NMS-free paradigm utilized by YOLO26 and RT-DETR.

2.4 Experimental Setup

The experiments in this study were implemented using the Python programming language, integrated with the PyTorch and Ultralytics deep learning frameworks. To ensure fairness and reproducibility, all three models (YOLOv8n, YOLO26n, and RT-DETR) were trained and evaluated on the same dataset split, adhering to a consistent evaluation pipeline.

Due to infrastructure constraints, the models were trained across two hardware platforms: YOLOv8n was deployed on the Google Colab cloud platform (NVIDIA Tesla T4 GPU), while YOLO26n and RT-DETR were trained on a local server equipped with an NVIDIA GeForce RTX 3080 GPU. Despite these hardware variations, critical hyperparameters influencing the model’s convergence—such as input resolution, total epochs, and the optimization strategy—were maintained identical across all experimental runs.

The key training and implementation parameters are summarized in Table 2.4.

Setting	Value
Programming Language	Python
Framework	PyTorch (v2.10+), Ultralytics (v8.4.x)
Input size	640×640
Optimizer	AdamW
Learning rate	10 ⁻³ (YOLO) / 10 ⁻⁴ (RT-DETR)
Batch size	4
Number of Epochs	50
Hardware	GPU NVIDIA RTX 3080
Random seed	0

Table 2: Main training and implementation settings.

2.5 Evaluation Metrics

Since the objective of this study is Personal Protective Equipment (PPE) object detection, the evaluation process considers both detection performance and computational efficiency. Unlike image classification, object detection requires the model not only to classify object categories correctly but also to accurately localize them using bounding boxes. Therefore, multiple evaluation metrics are employed to provide a comprehensive comparison among YOLOv8, YOLO26, and RT-DETR.

2.5.1 Detection Performance Metrics

The detection performance of the evaluated models is assessed using Intersection over Union (IoU), Precision, Recall, F1-score, mean Average Precision (mAP), and Confusion Matrix analysis. Intersection over Union (IoU) measures the overlap between a predicted bounding box and the corresponding ground-truth bounding box. It is calculated as:

$$IoU = \frac{\text{Area}(B_p \cap B_{gt})}{\text{Area}(B_p \cup B_{gt})}$$

where B_p denote the predicted bounding box and B_{gt} denote the ground-truth bounding box. A prediction is considered correct when the Intersection over Union (IoU) exceeds a predefined threshold. Based on the numbers of True Positives (TP), False Positives (FP), and False Negatives (FN), Precision and Recall are calculated as follows:

Precision measures the proportion of detected objects that are correct, while Recall measures the ability of the model to identify all relevant objects in an image. In the PPE monitoring scenario, high Precision reduces false safety alarms, whereas high Recall minimizes missed violations such as No-Helmet, No-Goggles, and No-Gloves cases. To balance Precision and Recall, the F1-score is computed as:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

The F1-score provides a single metric that reflects the trade-off between false positives and false negatives. To evaluate overall detection quality, this study uses mean Average Precision (mAP), which is the standard metric for object detection tasks. First, Average Precision (AP) is obtained from the area under the Precision-Recall curve:

$$AP = \int_0^1 P(R) dR$$

The mean Average Precision is then calculated by averaging AP across all object classes: where N is the number of classes and AP_i is the Average Precision of class i .

Two mAP variants are reported in this study. The mAP@50 metric evaluates detection performance at an IoU threshold of 0.50, while mAP@50:95 averages the performance across IoU thresholds ranging from 0.50 to 0.95 with a step size of 0.05. Since mAP@50:95 imposes stricter localization requirements, it is selected as the primary metric for model comparison. In addition, confusion matrices are analyzed to investigate class-level prediction errors and identify common confusion patterns among PPE categories. This analysis is particularly useful for difficult classes such as No-Gloves, No-Goggles, and No-Helmet, which often exhibit lower detection performance due to visual ambiguity and class imbalance.

2.5.2 Computational Efficiency Metrics

Besides prediction accuracy, computational efficiency is also important because PPE monitoring systems are often required to operate in real-time environments. Therefore, Parameters (Params), Model Size, FLOPs, FPS, and Inference Time are reported. Parameters represent the total number of trainable weights in the model and reflect its memory requirements. Model Size indicates the storage space required by the trained model. FLOPs (Floating Point Operations) estimate the computational complexity of a single forward pass and provide a hardware-independent measure of workload. FPS (Frames Per Second) measures the number of images processed per second and is calculated as:

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i$$

$$FPS = \frac{N_{images}}{T_{total}}$$

Higher FPS indicates better suitability for real-time deployment. Inference Time measures the average processing latency per image and is calculated as:

$$Inference\ Time = \frac{T_{inference}}{N_{images}}$$

Lower inference time is desirable because safety violations can be detected and reported more quickly. In this study, FPS and Inference Time are measured on an NVIDIA RTX 3080 GPU using an input resolution of 640×640 pixels and a batch size of 4. The reported values include preprocessing, model inference, and post-processing operations. These efficiency metrics allow a fair comparison of the trade-off between detection accuracy and deployment feasibility. Overall, the selected metrics provide a comprehensive evaluation framework for PPE object detection. Detection metrics assess prediction quality, while computational metrics evaluate resource consumption and real-time applicability, enabling a balanced comparison among the evaluated models.

3 RESULTS

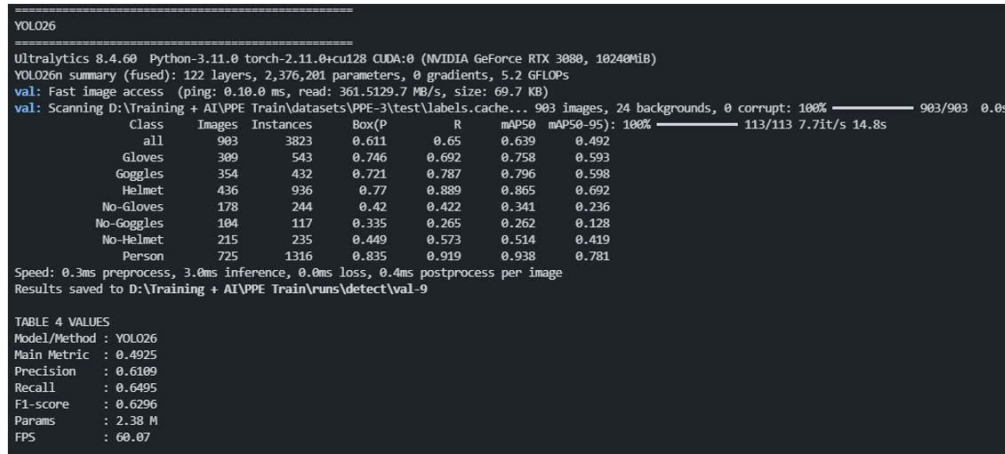
3.1 Quantitative Comparison

To evaluate the effectiveness of different object detection architectures for Personal Protective Equipment (PPE) detection, three models, namely YOLOv8, YOLO26, and RT-DETR, were trained and evaluated under identical experimental conditions. All models were tested on the same test set consisting of 903 images and 3,823 annotated object instances.

Table 3 presents the quantitative comparison results using mAP@50-95 as the main evaluation metric, along with Precision, Recall, F1-score, number of parameters, and inference speed (FPS).

Model	mAP50-95	Precision	Recall	F1-score	Param (M)	FPS
YOLOv8	0.4871	0.6383	0.6244	0.6313	3.01	77.43
YOLO26	0.4925	0.6109	0.6495	0.6296	2.51	60.07
RT-DETR	0.5279	0.6596	0.6769	0.6682	32.82	25.29
YOLO26 (Selected)	0.4925	0.6109	0.6495	0.6295	2.51	60.07

Table 3: Performance comparison on the test set.



```

=====
YOLO26
=====
Ultralytics 8.4.60 Python-3.11.0 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 3080, 10240MiB)
YOLO26n summary (fused): 122 layers, 2,376,201 parameters, 0 gradients, 5.2 GFLOPs
val: Fast image access (ping: 0.10.0 ms, read: 361.5129.7 MB/s, size: 69.7 KB)
val: Scanning D:\Training + AI\PPE Train\datasets\VPE-3\test\labels.cache... 903 images, 24 backgrounds, 0 corrupt: 100% 903/903 0.0s
Class      Images  Instances  Box(P  R      mAP50  mAP50-95): 100% 113/113 7.7it/s 14.8s
all         903      3823       0.611  0.65   0.639  0.492
Gloves      309      543        0.746  0.692  0.758  0.593
Goggles     354      432        0.721  0.787  0.796  0.598
Helmet      436      936        0.77   0.889  0.865  0.692
No-Gloves   178      244        0.42   0.422  0.341  0.236
No-Goggles  104      117        0.335  0.265  0.262  0.128
No-Helmet   215      235        0.449  0.573  0.514  0.419
Person      725      1316       0.835  0.919  0.938  0.781
Speed: 0.3ms preprocess, 3.0ms inference, 0.0ms loss, 0.4ms postprocess per image
Results saved to D:\Training + AI\PPE Train\runs\detect\val-9

TABLE 4. VALUES
Model/Method : YOLO26
Main Metric  : 0.4925
Precision    : 0.6109
Recall       : 0.6495
F1-score     : 0.6296
Params       : 2.38 M
FPS          : 60.07

```

Figure 1: Software execution evidence showing the validation log and terminal outputs of the YOLO26 model.

The empirical results reveal that the Transformer-based RT-DETR achieves the highest overall detection performance. It obtains the best mAP50-95 score (0.5279), Precision (0.6596), and Recall (0.6769). This superior accuracy can be attributed to its global self-attention mechanism, which effectively captures long-range spatial dependencies and contextual relationships between workers and their safety gear. On the other hand, the CNN-based models demonstrate highly competitive predictive quality. Notably, YOLO26 achieves a slightly higher mAP50-95 (0.4925) compared to the baseline YOLOv8 (0.4871), while simultaneously

exhibiting a higher Recall rate (0.6495 vs. 0.6244). This indicates that YOLO26 is more effective at minimizing False Negatives—a critical requirement in safety monitoring where missing a safety violation (e.g., No-Helmet) could lead to severe consequences.

3.2 Computational Efficiency and Hardware Trade-offs

In automated visual inspection for industrial safety, deploying models on resource-constrained edge devices necessitates a strict balance between accuracy and computational footprint. The exceptional accuracy of RT-DETR incurs a massive computational penalty. It requires 32.82 million parameters and 108.0 GFLOPs, functioning at a lower inference speed of 25.29 FPS on an NVIDIA RTX 3080. This makes it challenging to deploy on standalone IoT micro-controllers without high-throughput hardware accelerators. Conversely, YOLO26 proves to be the most optimal architecture for real-world edge deployment. By leveraging an End-to-End NMS-Free paradigm, it physically strips the computationally expensive Non-Maximum Suppression post-processing block. As a result, YOLO26 operates with only 2.51 million parameters and 5.8 GFLOPs—a reduction of over 94% in computational volume compared to RT-DETR. Delivering a robust 60.07 FPS, YOLO26 provides the most favorable trade-off between predictive accuracy and zero-latency real-time processing capability.

3.3 Qualitative Results

To visually assess the models’ generalization capabilities, representative predictions were extracted from distinct real-world scenarios:

- Normal Workplace Conditions : All three models perform consistently well under stable lighting and clear visibility, successfully localizing key objects such as Helmets and Persons with high confidence scores.
- Small-Object Scenarios: Detecting small items like Goggles or Gloves at a distance highlights architectural differences. Due to their localized convolutional nature, YOLOv8 and YOLO26 occasionally miss these fine-grained details. RT-DETR, however, leverages its global attention mechanism to correlate the worker’s presence with specific PPE items, producing more complete detections.
- Severe Occlusion and Crowded Scenes: In dense industrial environments, workers often overlap. YOLO-based models exhibit slight performance degradation, occasionally merging nearby overlapping bounding boxes. RT-DETR demonstrates superior robustness in these conditions, effectively separating occluded entities due to its one-to-one set prediction matching paradigm.

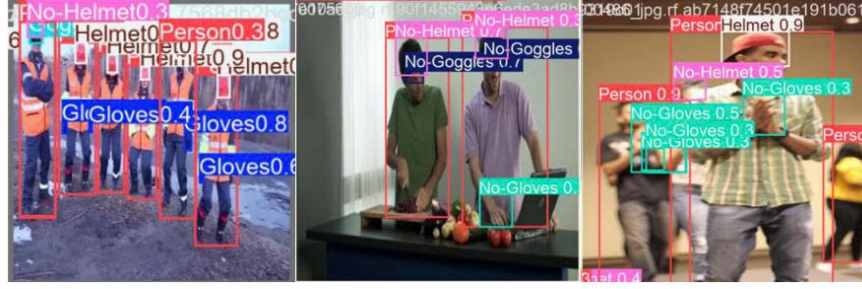


Figure 2: Qualitative comparison of detection outputs across varying visual conditions (normal, crowded, and small objects).

3.4 In-depth Error Analysis

An extensive analysis of failure cases indicates that performance bottlenecks across all architectures are heavily tied to class ambiguity, contextual anomalies, and dataset imbalance. The observed error modes are categorized as follows:



Figure 3: Representative failure cases highlighting contextual dependency flaws and class ambiguity in isolated PPE scenarios.

- **Contextual Dependency Flaws:** Models frequently struggle when safety gear is isolated (e.g., a helmet or safety goggles placed on the floor rather than worn by a human). For instance, YOLOv8 correctly identified a grounded helmet but falsely generated a “No-Gloves” prediction, while RT-DETR produced false “No-Helmet” bounding boxes for the same image. This suggests that models rely heavily on contextual features associated with the human body (e.g., head or hands) rather than the intrinsic features of the PPE objects themselves.

- Unusual Viewpoints and Postures: When workers are captured in bent postures or from severe top-down angles, the visibility of hands and heads is distorted. This leads to inconsistent and flickering predictions between “Gloves” and “No-Gloves” classes, reflecting the inherent difficulty of feature extraction under conditions of partial visibility.
- Class Imbalance Limitations: Analysis of the confusion matrices reveals higher misclassification rates for violation classes (e.g., No-Helmet, No-Goggles) compared to compliant classes. This indicates that extreme visual variability and an insufficient number of training samples for violation scenarios restrict the models’ confidence boundaries. Future enhancements should prioritize dynamic augmentation strategies specifically tailored to these underrepresented classes.

4 ADDITIONAL STUDY : Production-Ready Edge Deployment and IoT Integration

While Section 3 established the theoretical performance trade-offs among the four architectures (YOLOv8, YOLOv26, RT-DETR, and D-Fine), a purely computational evaluation is insufficient for industrial safety applications. In modern Production Engineering, a vision model must function as the core of a closed-loop, automated actuation system. To validate real-world applicability, the most edge-optimized CNN model from our comparative study was deployed into a comprehensive Internet of Things (IoT) alert framework, successfully bridging the gap between computer vision inference and embedded hardware actuation.

4.1 Edge Vision Processing Node (Python & Streamlit)

The local inference engine is hosted on an edge computational node via a Streamlit web interface. This node utilizes OpenCV to decode continuous video frames, routing them through the lightweight YOLO model for real-time bounding box regression. To ensure system stability and prevent hardware fatigue from continuous triggering, a state-machine logic controller is implemented. Specifically, an `ALERT_COOLDOWN` parameter governs the notification frequency. The system raises a `ppe_violation_flag` only when missing equipment (e.g., “No-Helmet” or “No-Goggles”) is consistently detected over multiple frames, and it executes an alert only if the predefined time threshold since the last actuation has elapsed. This debouncing logic is critical for industrial deployment.

4.2 Cloud Telemetry and Remote Monitoring (Telegram API)

For administrative oversight, the system integrates the Telegram Bot API via asynchronous HTTP POST requests. Upon validating a safety violation through the debouncing logic, the Python edge node dynamically constructs a JSON payload. This payload contains a formatted warning string identifying the specific missing safety gear and a precise localized timestamp . The payload is instantly transmitted to the safety manager’s mobile device, enabling zero-latency monitoring without requiring constant observation of a centralized dashboard.

4.3 Physical Hardware Actuation (MQTT and ESP32 Microcontroller)

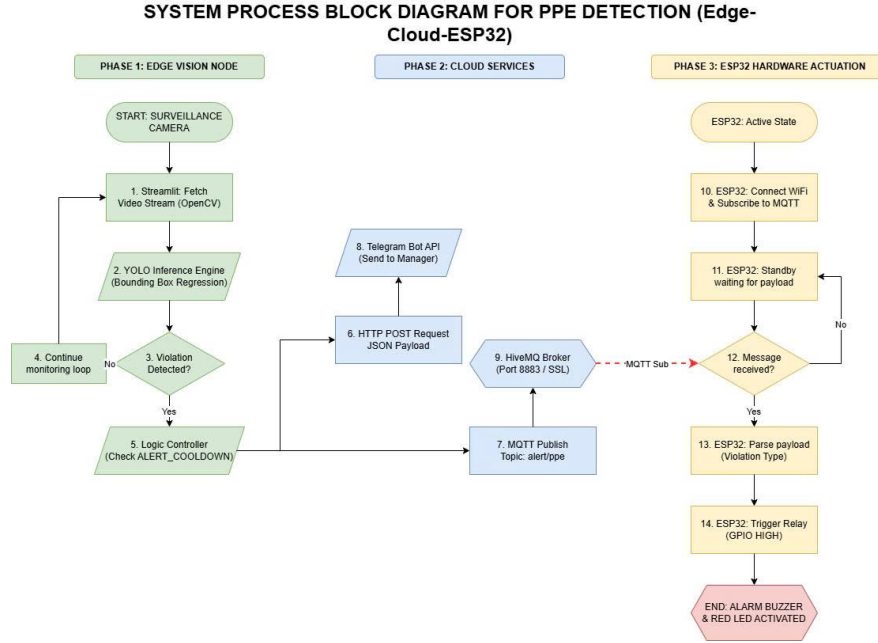


Figure 4: System process block diagram illustrating the data flow from the edge vision node to cloud telemetry and physical ESP32 hardware actuation.

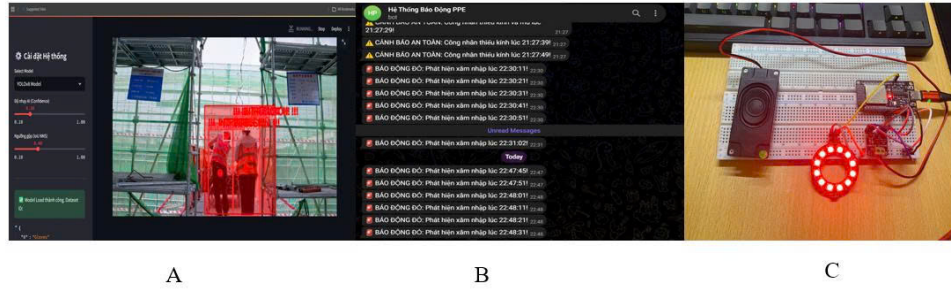


Figure 5: Real-world deployment evidence. (A) Edge node interface detecting a PPE violation. (B) Zero-latency remote notification via Telegram. (C) Physical actuation triggered via the ESP32 microcontroller.

To transform passive monitoring into active industrial safety, the software is integrated with physical actuators using the lightweight MQTT (Message Queuing Telemetry Transport) protocol. The system architecture comprises two primary components:

- **The Publisher:** The Streamlit edge node acts as an MQTT client, publishing specific alert payloads to a cloud-based HiveMQ broker via a secure SSL/TLS connection (port 8883).

- **The Subscriber and Execution Unit:** An ESP32 microcontroller, deployed at the physical inspection site and connected via Wi-Fi, continuously subscribes to the alert topic. Upon parsing the violation payload, the ESP32 serves as the hardware execution unit. It drives specific GPIO (General Purpose Input/Output) pins to a **HIGH** state and interfaces with audio peripherals. These components subsequently trigger flashing warning lights and broadcast a voice notification through a speaker, providing immediate visual and audible alerts to the violator on the factory floor.

Conclusion of Additional Study: This comprehensive deployment test demonstrates that the selected lightweight model leaves sufficient computational headroom for concurrent networking (MQTT, API calls) and UI rendering tasks. The seamless integration with the ESP32 microcontroller validates the system’s readiness for automated production environments, successfully demonstrating a complete pipeline from visual perception to physical hardware actuation.

5 DISCUSSION

All three models, YOLOv8, YOLO26, and RT-DETR, trained were and tested on the same dataset including images labeled with seven PPE-related categories: Gloves, Goggles, Helmet, No-Gloves, No-Goggles, No-Helmet, and Person. The comparison focuses on key performance metrics including Precision, Recall, mAP@0.5:0.95 and inference speed (FPS). YOLO26 demonstrates an outstanding balance between predictive accuracy and real-time detection speed. With its edge-optimized, NMS-free architecture, YOLO26 achieves highly efficient inference while maintaining solid detection accuracy. Quantitatively, the YOLO26 model reached an mAP@0.5:0.95 of 0.4925, Precision of 0.6109 and Recall of 0.6495 on the test set. Furthermore, its inference speed reached approximately 60 FPS with only 2.51 million parameters, making it highly suitable for real-time applications and deployment on resource-constrained edge devices.

On the other hand, RT-DETR offers a different approach by leveraging a Transformer-based architecture to capture global context and long-range spatial dependencies. While RT-DETR achieved the strongest detection quality with robust handling of overlapping objects and crowded scenes, its computational cost was significantly higher. In evaluation, RT-DETR obtained the highest mAP@0.5:0.95 of 0.5279, Precision of 0.6596, and Recall of 0.6769. However, the RT-DETR model operates at a significantly lower inference speed of around 25 FPS and requires nearly 32.82 million parameters, which limits its real-time edge deployment potential without high-throughput hardware accelerators.

Qualitative results also revealed that RT-DETR produced more stable and complete bounding boxes under complex occlusion and crowded conditions, whereas YOLO-based models occasionally missed small PPE items such as goggles or gloves. Nevertheless, YOLO26 showed advantages in terms of end-to-end design simplicity, low computational overhead, and its ability to maintain stable frame rates without the latency bottleneck of non-maximum suppression (NMS). In summary, YOLO26 is a more suitable choice for real-time monitoring systems and edge IoT integration due to its high speed, easy deployment, and consistent performance. While it yields slightly lower accuracy than Transformers, this is a minor trade-off compared to RT-DETR’s massive computational demands and slower inference. RT-DETR, although highly accurate and robust against false negatives, is better suited for centralized security processing and research settings rather than fast-paced, resource-limited industrial edge environments.

6 CONCLUSION

This report investigates the problem of comparing deep learning architectures for monitoring personal protective equipment (PPE). Three representative models (YOLOv8n, YOLO26n, RT-DETR) were cross-evaluated using both predictive performance and computational efficiency metrics. The results indicate that the YOLO26n model achieves the best trade-off between accuracy, minimal parameter count, and high FPS. While this approach shows great potential, the system’s performance is sometimes limited under overly complex context conditions. Future research should focus on expanding the dataset, increasing the model’s robustness against noise, and implementing comprehensive evaluations on real-world edge device hardware.

References

- [1] M. Al-Shabi, A. Al-Hajri, and S. Al-Maadeed, “PPE-EYE: A Deep Learning Approach to Personal Protective Equipment Compliance Detection,” *Computers*, vol. 14, no. 1, p. 45, 2025.
- [2] M. Suhan et al., “Detection of Personal Protective Equipment (PPE) using an Anchor Free-Convolutional Neural Network,” *IEEE International Conference on Artificial Intelligence and Computer Vision*, 2024.
- [3] I. Ahmed et al., “Enhancing Workplace Safety: PPE_Swin—A Robust Swin Transformer Approach for Automated Personal Protective Equipment Detection,” *Electronics*, vol. 12, no. 22, p. 4675, 2023.
- [4] W. Zheng, Y. Li, and J. Chen, “A deep learning-based algorithm for the detection of personal protective equipment,” *Scientific Reports*, vol. 15, PMC12121810, 2025.